

Time: 3 hr

Number of Questions: 300

Q1. In production-quality Python code, which statements about Swapping and multiple assignment are valid? Select all that apply.

- A. Set ordering should be relied upon for stable business logic output.
- B. Tuples allow in-place item reassignment because they are sequence types.
- C. A single-item tuple requires a trailing comma.
- D. Tuple hashability depends on all contained elements being hashable.

Q2. While applying Common pitfalls, which statements are technically correct? Select all that apply.

- A. Collection choice should be based on access pattern, mutation pattern, and complexity tradeoffs.
- B. Deep copies and shallow copies behave identically for nested mutable structures.
- C. Dictionary keys can safely be mutable lists as long as values are immutable.
- D. Readability and deterministic output requirements can be more important than micro-optimizations.

Q3. Which statements are correct about In-Place Set Updates vs New Set Creation in Python? Select all that apply.

- A. Sets support index-based access like lists for deterministic element lookup.
- B. Readability and deterministic output requirements can be more important than micro-optimizations.
- C. Using the correct collection type often removes the need for complex conditional logic.
- D. Deep copies and shallow copies behave identically for nested mutable structures.

Q4. In production-quality Python code, which statements about Loop unpacking are valid? Select all that apply.

- A. Dictionary keys can safely be mutable lists as long as values are immutable.
- B. A single-item tuple requires a trailing comma.
- C. Shallow copies duplicate only the outer container and may share nested mutable objects.
- D. Any tuple can be a dictionary key even when it contains lists or dicts.

Q5. In production-quality Python code, which statements about __missing__ Hook and Custom Mapping Behavior are valid? Select all that apply.

- A. Calling `dict.get()` raises `KeyError` when a key is missing.
- B. Merging dictionaries with duplicate keys keeps the right-side value.
- C. Dictionary view objects (`keys`, `items`) are dynamic and reflect later dictionary updates.
- D. Set ordering should be relied upon for stable business logic output.

Q6. Which statements are correct about Tuple Mastery Checklist in Python? Select all that apply.

- A. Dictionary keys can safely be mutable lists as long as values are immutable.
- B. Using the correct collection type often removes the need for complex conditional logic.
- C. Any tuple can be a dictionary key even when it contains lists or dicts.
- D. Collection choice should be based on access pattern, mutation pattern, and complexity tradeoffs.

Q7. While applying `clear()` (remove all items), which statements are technically correct? Select all that apply.

- A. List front-pop operations (`pop(0)`) are always $O(1)$, so they are ideal for queue workloads.
- B. Sets support index-based access like lists for deterministic element lookup.
- C. Using the correct collection type often removes the need for complex conditional logic.
- D. Readability and deterministic output requirements can be more important than micro-optimizations.

Q8. Which statements are correct about What Is a List in Python? Select all that apply.

- A. Collection choice should be based on access pattern, mutation pattern, and complexity tradeoffs.
- B. Using the correct collection type often removes the need for complex conditional logic.
- C. List front-pop operations (`pop(0)`) are always $O(1)$, so they are ideal for queue workloads.
- D. Using `list.sort()` returns a new sorted list and leaves the original untouched.

Q9. While applying Copy and In-Place Updates, which statements are technically correct? Select all that apply.

- A. Deep copies and shallow copies behave identically for nested mutable structures.
- B. Set ordering should be relied upon for stable business logic output.
- C. Collection choice should be based on access pattern, mutation pattern, and complexity tradeoffs.
- D. Set algebra operators (`|`, `&`, `-`, `^`) model data comparison tasks clearly.

Q10. Which statements are correct about Add One Element: `add()` in Python? Select all that apply.

- A. Dictionary keys can safely be mutable lists as long as values are immutable.
- B. Readability and deterministic output requirements can be more important than micro-optimizations.
- C. Sets support index-based access like lists for deterministic element lookup.
- D. Set algebra operators (`|`, `&`, `-`, `^`) model data comparison tasks clearly.

Q11. Predict the exact single-line output of the following code.

```
values = [10, 6, 6, 10]
unique_ordered = list(dict.fromkeys(values))
print(sum(unique_ordered))
```

Answer: _____

Q12. Predict the exact single-line output of the following code.

```
numbers = [8, 6, 1, 6]
numbers.sort(reverse=True)
print(numbers[0] - numbers[-1])
```

Answer: _____

Q13. Predict the exact single-line output of the following code.

```
records = [2, 12, 2, 2]
frequency = {}
for value in records:
    frequency[value] = frequency.get(value, 0) + 1
print(frequency[2])
```

Answer: _____

Q14. Predict the exact single-line output of the following code.

```
values = [6, 4, 5, 6]
unique_ordered = list(dict.fromkeys(values))
print(sum(unique_ordered))
```

Answer: _____

Q15. Predict the exact single-line output of the following code.

```
values = [10, 11, 5, 10]
unique_ordered = list(dict.fromkeys(values))
print(sum(unique_ordered))
```

Answer: _____

Q16. Predict the exact single-line output of the following code.

```
pair_list = [(7, 5), (4, 4), (7, 4)]
total = 0
for left, right in pair_list:
    total += left - right
print(total)
```

Answer: _____

Q17. Predict the exact single-line output of the following code.

```
values = [5, 8, 6, 5]
unique_ordered = list(dict.fromkeys(values))
print(sum(unique_ordered))
```

Answer: _____

Q18. Predict the exact single-line output of the following code.

```
values = [8, 5, 1, 8]
unique_ordered = list(dict.fromkeys(values))
print(sum(unique_ordered))
```

Answer: _____

Q19. Predict the exact single-line output of the following code.

```
records = [5, 9, 5, 4]
frequency = {}
for value in records:
    frequency[value] = frequency.get(value, 0) + 1
print(frequency[5])
```

Answer: _____

Q20. Predict the exact single-line output of the following code.

```
records = [4, 11, 4, 1]
frequency = {}
for value in records:
    frequency[value] = frequency.get(value, 0) + 1
print(frequency[4])
```

Answer: _____

Q21. Predict the exact single-line output of the following code.

```
records = [6, 8, 6, 1]
frequency = {}
for value in records:
    frequency[value] = frequency.get(value, 0) + 1
print(frequency[6])
```

Answer: _____

Q22. Predict the exact single-line output of the following code.

```
numbers = [6, 10, 1, 2]
numbers.sort(reverse=True)
print(numbers[0] - numbers[-1])
```

Answer: _____

Q23. Predict the exact single-line output of the following code.

```
records = [6, 9, 6, 3]
frequency = {}
for value in records:
    frequency[value] = frequency.get(value, 0) + 1
print(frequency[6])
```

Answer: _____

Q24. Predict the exact single-line output of the following code.

```
left_ids = {2, 12, 4}
right_ids = {12, 3}
shared = left_ids & right_ids
print(sum(shared))
```

Answer: _____

Q25. Predict the exact single-line output of the following code.

```
left_ids = {2, 12, 6}
right_ids = {12, 9}
shared = left_ids & right_ids
print(sum(shared))
```

Answer: _____

Q26. Predict the exact single-line output of the following code.

```
pair_list = [(8, 8), (4, 8), (8, 4)]
total = 0
for left, right in pair_list:
    total += left - right
print(total)
```

Answer: _____

Q27. Predict the exact single-line output of the following code.

```
records = [5, 7, 5, 1]
frequency = {}
for value in records:
    frequency[value] = frequency.get(value, 0) + 1
print(frequency[5])
```

Answer: _____

Q28. Predict the exact single-line output of the following code.

```
values = [10, 3, 3, 10]
unique_ordered = list(dict.fromkeys(values))
print(sum(unique_ordered))
```

Answer: _____

Q29. Predict the exact single-line output of the following code.

```
pair_list = [(6, 7), (6, 9), (6, 6)]
total = 0
for left, right in pair_list:
    total += left - right
print(total)
```

Answer: _____

Q30. Predict the exact single-line output of the following code.

```
left_ids = {7, 4, 6}
right_ids = {4, 7}
shared = left_ids & right_ids
print(sum(shared))
```

Answer: _____

Q31. Predict the exact single-line output of the following code.

```
records = [10, 9, 10, 2]
frequency = {}
for value in records:
    frequency[value] = frequency.get(value, 0) + 1
print(frequency[10])
```

Answer: _____

Q32. Predict the exact single-line output of the following code.

```
numbers = [10, 6, 5, 3]
numbers.sort(reverse=True)
print(numbers[0] - numbers[-1])
```

Answer: _____

Q33. Predict the exact single-line output of the following code.

```
records = [11, 10, 11, 2]
frequency = {}
for value in records:
    frequency[value] = frequency.get(value, 0) + 1
print(frequency[11])
```

Answer: _____

Q34. Predict the exact single-line output of the following code.

```
pair_list = [(7, 10), (6, 4), (7, 6)]
total = 0
for left, right in pair_list:
    total += left - right
print(total)
```

Answer: _____

Q35. Predict the exact single-line output of the following code.

```
values = [2, 5, 1, 2]
unique_ordered = list(dict.fromkeys(values))
print(sum(unique_ordered))
```

Answer: _____

Q36. Which statements are correct about When Tuple Harms Readability in Python? Select all that apply.

- A. Shallow copies duplicate only the outer container and may share nested mutable objects.
- B. Set ordering should be relied upon for stable business logic output.
- C. Collection choice should be based on access pattern, mutation pattern, and complexity tradeoffs.
- D. Dictionary keys can safely be mutable lists as long as values are immutable.

Q37. While applying Internal behavior (conceptual), which statements are technically correct? Select all that apply.

- A. Dictionary view objects (``keys``, ``items``) are dynamic and reflect later dictionary updates.
- B. Dictionary keys can safely be mutable lists as long as values are immutable.
- C. Calling ``dict.get()`` raises ``KeyError`` when a key is missing.
- D. Using the correct collection type often removes the need for complex conditional logic.

Q38. Which statements are correct about Interview-Level Mental Model in Python? Select all that apply.

- A. Shallow copies duplicate only the outer container and may share nested mutable objects.
- B. Using the correct collection type often removes the need for complex conditional logic.
- C. Deep copies and shallow copies behave identically for nested mutable structures.
- D. Set ordering should be relied upon for stable business logic output.

Q39. In production-quality Python code, which statements about Self-check questions (with verification output) are valid? Select all that apply.

- A. Collection choice should be based on access pattern, mutation pattern, and complexity tradeoffs.
- B. Dictionary keys can safely be mutable lists as long as values are immutable.
- C. List front-pop operations (``pop(0)``) are always $O(1)$, so they are ideal for queue workloads.
- D. Using the correct collection type often removes the need for complex conditional logic.

Q40. While applying Production Checklist for Dictionary Usage, which statements are technically correct? Select all that apply.

- A. Readability and deterministic output requirements can be more important than micro-optimizations.
- B. Collection choice should be based on access pattern, mutation pattern, and complexity tradeoffs.
- C. Calling ``dict.get()`` raises ``KeyError`` when a key is missing.
- D. Deep copies and shallow copies behave identically for nested mutable structures.

Q41. While applying Hashing, Equality, and Key Design (Deep Interview Topic), which statements are technically correct? Select all that apply.

- A. List front-pop operations (``pop(0)``) are always $O(1)$, so they are ideal for queue workloads.
- B. Readability and deterministic output requirements can be more important than micro-optimizations.
- C. Shallow copies duplicate only the outer container and may share nested mutable objects.
- D. Dictionary iteration order in modern Python is random and should never be expected.

Q42. Which statements are correct about Indexing and Slicing in Python? Select all that apply.

- A. Readability and deterministic output requirements can be more important than micro-optimizations.
- B. Using ``list.sort()`` returns a new sorted list and leaves the original untouched.
- C. Collection choice should be based on access pattern, mutation pattern, and complexity tradeoffs.
- D. Dictionary keys can safely be mutable lists as long as values are immutable.

Q43. In production-quality Python code, which statements about Production Dict Checklist (Advanced) are valid? Select all that apply.

- A. Using the correct collection type often removes the need for complex conditional logic.
- B. Set ordering should be relied upon for stable business logic output.
- C. Readability and deterministic output requirements can be more important than micro-optimizations.
- D. Dictionary keys can safely be mutable lists as long as values are immutable.

Q44. While applying Membership testing, which statements are technically correct? Select all that apply.

- A. Dictionary view objects (`keys`, `items`) are dynamic and reflect later dictionary updates.
- B. Collection choice should be based on access pattern, mutation pattern, and complexity tradeoffs.
- C. Calling `dict.get()` raises `KeyError` when a key is missing.
- D. Dictionary keys can safely be mutable lists as long as values are immutable.

Q45. While applying Grouping with setdefault vs defaultdict, which statements are technically correct? Select all that apply.

- A. Dictionary view objects (`keys`, `items`) are dynamic and reflect later dictionary updates.
- B. Dictionary keys can safely be mutable lists as long as values are immutable.
- C. Set ordering should be relied upon for stable business logic output.
- D. Collection choice should be based on access pattern, mutation pattern, and complexity tradeoffs.

Q46. Predict the exact single-line output of the following code.

```
data = [2, 10, 6, 2, 2, 6, 10]
freq = {}
for item in data:
    freq[item] = freq.get(item, 0) + 1
print(sum(key * value for key, value in freq.items()))
```

Answer: _____

Q47. Predict the exact single-line output of the following code.

```
data = [3, 12, 7, 4, 3, 7, 12]
freq = {}
for item in data:
    freq[item] = freq.get(item, 0) + 1
print(sum(key * value for key, value in freq.items()))
```

Answer: _____

Q48. Predict the exact single-line output of the following code.

```
rows = [[3, 7], [7, 5], [10, 12]]
columns = list(zip(*rows))
print(sum(columns[0]) - sum(columns[1]))
```

Answer: _____

Q49. Predict the exact single-line output of the following code.

```
pairs = [(5, 6), (5, 6), (6, 6), (5, 6), (6, 6)]
grouped = {}
for k, v in pairs:
    grouped.setdefault(k, []).append(v)
print(sum(len(vs) for vs in grouped.values()) + len(grouped))
```

Answer: _____

Q50. Predict the exact single-line output of the following code.

```
data = [10, 9, 1, 4, 10, 1, 9]
freq = {}
for item in data:
    freq[item] = freq.get(item, 0) + 1
print(sum(key * value for key, value in freq.items()))
```

Answer: _____

Q51. Predict the exact single-line output of the following code.

```
pairs = [(11, 12), (11, 3), (12, 7), (11, 7), (3, 7)]
grouped = {}
for k, v in pairs:
    grouped.setdefault(k, []).append(v)
print(sum(len(vs) for vs in grouped.values()) + len(grouped))
```

Answer: _____

Q52. Predict the exact single-line output of the following code.

```
data = [5, 10, 1, 5, 5, 1, 10]
freq = {}
for item in data:
    freq[item] = freq.get(item, 0) + 1
print(sum(key * value for key, value in freq.items()))
```

Answer: _____

Q53. Predict the exact single-line output of the following code.

```
pairs = [(10, 3), (10, 2), (3, 5), (10, 5), (2, 5)]
grouped = {}
for k, v in pairs:
    grouped.setdefault(k, []).append(v)
print(sum(len(vs) for vs in grouped.values()) + len(grouped))
```

Answer: _____

Q54. Predict the exact single-line output of the following code.

```
matrix = [[3, 8, 3], [5, 5, 5], [8, 3, 5]]
flat = [x for row in matrix for x in row]
unique = sorted(set(flat))
index = {value: idx for idx, value in enumerate(unique)}
print(sum(index[x] for x in flat[:5]))
```

Answer: _____

Q55. Predict the exact single-line output of the following code.

```
rows = [[7, 3], [4, 6], [11, 9]]
columns = list(zip(*rows))
print(sum(columns[0]) - sum(columns[1]))
```

Answer: _____

Q56. Predict the exact single-line output of the following code.

```
matrix = [[11, 12, 11], [5, 7, 5], [12, 11, 7]]
flat = [x for row in matrix for x in row]
unique = sorted(set(flat))
index = {value: idx for idx, value in enumerate(unique)}
print(sum(index[x] for x in flat[:5]))
```

Answer: _____

Q57. Predict the exact single-line output of the following code.

```
matrix = [[4, 5, 4], [3, 6, 3], [5, 4, 6]]
flat = [x for row in matrix for x in row]
unique = sorted(set(flat))
index = {value: idx for idx, value in enumerate(unique)}
print(sum(index[x] for x in flat[:5]))
```

Answer: _____

Q58. Predict the exact single-line output of the following code.

```
data = [7, 5, 6, 4, 7, 6, 5]
freq = {}
for item in data:
    freq[item] = freq.get(item, 0) + 1
print(sum(key * value for key, value in freq.items()))
```

Answer: _____

Q59. Predict the exact single-line output of the following code.

```
matrix = [[3, 4, 3], [6, 9, 6], [4, 3, 9]]
flat = [x for row in matrix for x in row]
unique = sorted(set(flat))
index = {value: idx for idx, value in enumerate(unique)}
print(sum(index[x] for x in flat[:5]))
```

Answer: _____

Q60. Predict the exact single-line output of the following code.

```
data = [9, 5, 4, 6, 9, 4, 5]
freq = {}
for item in data:
    freq[item] = freq.get(item, 0) + 1
print(sum(key * value for key, value in freq.items()))
```

Answer: _____

Q61. Predict the exact single-line output of the following code.

```
words = ['k5', 'k12', 'k5', 'k3', 'k12', 'k3']
rank = {name: i for i, name in enumerate(sorted(set(words)), start=1)}
counts = {}
for name in words:
    counts[name] = counts.get(name, 0) + rank[name]
print(sum(counts.values()))
```

Answer: _____

Q62. Predict the exact single-line output of the following code.

```
rows = [[9, 10], [3, 3], [12, 13]]
columns = list(zip(*rows))
print(sum(columns[0]) - sum(columns[1]))
```

Answer: _____

Q63. Predict the exact single-line output of the following code.

```
pairs = [(11, 6), (11, 4), (6, 5), (11, 5), (4, 5)]
grouped = {}
for k, v in pairs:
    grouped.setdefault(k, []).append(v)
print(sum(len(vs) for vs in grouped.values()) + len(grouped))
```

Answer: _____

Q64. Predict the exact single-line output of the following code.

```
pairs = [(5, 12), (5, 4), (12, 8), (5, 8), (4, 8)]
grouped = {}
for k, v in pairs:
    grouped.setdefault(k, []).append(v)
print(sum(len(vs) for vs in grouped.values()) + len(grouped))
```

Answer: _____

Q65. Predict the exact single-line output of the following code.

```
words = ['k4', 'k3', 'k4', 'k4', 'k3', 'k6']
rank = {name: i for i, name in enumerate(sorted(set(words)), start=1)}
counts = {}
for name in words:
    counts[name] = counts.get(name, 0) + rank[name]
print(sum(counts.values()))
```

Answer: _____

Q66. Predict the exact single-line output of the following code.

```
matrix = [[8, 9, 8], [1, 6, 1], [9, 8, 6]]
flat = [x for row in matrix for x in row]
unique = sorted(set(flat))
index = {value: idx for idx, value in enumerate(unique)}
print(sum(index[x] for x in flat[:5]))
```

Answer: _____

Q67. Predict the exact single-line output of the following code.

```
pairs = [(7, 9), (7, 4), (9, 2), (7, 2), (4, 2)]
grouped = {}
for k, v in pairs:
    grouped.setdefault(k, []).append(v)
print(sum(len(vs) for vs in grouped.values()) + len(grouped))
```

Answer: _____

Q68. Predict the exact single-line output of the following code.

```
words = ['k9', 'k5', 'k9', 'k6', 'k5', 'k9']
rank = {name: i for i, name in enumerate(sorted(set(words)), start=1)}
counts = {}
for name in words:
    counts[name] = counts.get(name, 0) + rank[name]
print(sum(counts.values()))
```

Answer: _____

Q69. Predict the exact single-line output of the following code.

```
words = ['k3', 'k7', 'k3', 'k2', 'k7', 'k4']
rank = {name: i for i, name in enumerate(sorted(set(words)), start=1)}
counts = {}
for name in words:
    counts[name] = counts.get(name, 0) + rank[name]
print(sum(counts.values()))
```

Answer: _____

Q70. Predict the exact single-line output of the following code.

```
matrix = [[6, 3, 6], [7, 4, 7], [3, 6, 4]]
flat = [x for row in matrix for x in row]
unique = sorted(set(flat))
index = {value: idx for idx, value in enumerate(unique)}
print(sum(index[x] for x in flat[:5]))
```

Answer: _____

Q71. Predict the exact single-line output of the following code.

```
words = ['k4', 'k7', 'k4', 'k6', 'k7', 'k5']
rank = {name: i for i, name in enumerate(sorted(set(words)), start=1)}
counts = {}
for name in words:
    counts[name] = counts.get(name, 0) + rank[name]
print(sum(counts.values()))
```

Answer: _____

Q72. Predict the exact single-line output of the following code.

```
words = ['k10', 'k5', 'k10', 'k2', 'k5', 'k6']
rank = {name: i for i, name in enumerate(sorted(set(words)), start=1)}
counts = {}
for name in words:
    counts[name] = counts.get(name, 0) + rank[name]
print(sum(counts.values()))
```

Answer: _____

Q73. Predict the exact single-line output of the following code.

```
rows = [[4, 9], [5, 3], [9, 12]]
columns = list(zip(*rows))
print(sum(columns[0]) - sum(columns[1]))
```

Answer: _____

Q74. Predict the exact single-line output of the following code.

```
pairs = [(8, 5), (8, 2), (5, 3), (8, 3), (2, 3)]
grouped = {}
for k, v in pairs:
    grouped.setdefault(k, []).append(v)
print(sum(len(vs) for vs in grouped.values()) + len(grouped))
```

Answer: _____

Q75. Predict the exact single-line output of the following code.

```
rows = [[4, 8], [1, 9], [5, 17]]
columns = list(zip(*rows))
print(sum(columns[0]) - sum(columns[1]))
```

Answer: _____

Q76. Predict the exact single-line output of the following code.

```
pairs = [(4, 11), (4, 2), (11, 8), (4, 8), (2, 8)]
grouped = {}
for k, v in pairs:
    grouped.setdefault(k, []).append(v)
print(sum(len(vs) for vs in grouped.values()) + len(grouped))
```

Answer: _____

Q77. Predict the exact single-line output of the following code.

```
data = [5, 4, 6, 8, 5, 6, 4]
freq = {}
for item in data:
    freq[item] = freq.get(item, 0) + 1
print(sum(key * value for key, value in freq.items()))
```

Answer: _____

Q78. Predict the exact single-line output of the following code.

```
rows = [[9, 8], [7, 8], [16, 16]]
columns = list(zip(*rows))
print(sum(columns[0]) - sum(columns[1]))
```

Answer: _____

Q79. Predict the exact single-line output of the following code.

```
pairs = [(4, 4), (4, 4), (4, 6), (4, 6), (4, 6)]
grouped = {}
for k, v in pairs:
    grouped.setdefault(k, []).append(v)
print(sum(len(vs) for vs in grouped.values()) + len(grouped))
```

Answer: _____

Q80. Predict the exact single-line output of the following code.

```
words = ['k5', 'k7', 'k5', 'k1', 'k7', 'k9']
rank = {name: i for i, name in enumerate(sorted(set(words)), start=1)}
counts = {}
for name in words:
    counts[name] = counts.get(name, 0) + rank[name]
print(sum(counts.values()))
```

Answer: _____

Q81. Predict the exact single-line output of the following code.

```
pairs = [(4, 8), (4, 6), (8, 9), (4, 9), (6, 9)]
grouped = {}
for k, v in pairs:
    grouped.setdefault(k, []).append(v)
print(sum(len(vs) for vs in grouped.values()) + len(grouped))
```

Answer: _____

Q82. Predict the exact single-line output of the following code.

```
data = [7, 11, 7, 9, 7, 7, 11]
freq = {}
for item in data:
    freq[item] = freq.get(item, 0) + 1
print(sum(key * value for key, value in freq.items()))
```

Answer: _____

Q83. Predict the exact single-line output of the following code.

```
matrix = [[2, 8, 2], [6, 6, 6], [8, 2, 6]]
flat = [x for row in matrix for x in row]
unique = sorted(set(flat))
index = {value: idx for idx, value in enumerate(unique)}
print(sum(index[x] for x in flat[:5]))
```

Answer: _____

Q84. Predict the exact single-line output of the following code.

```
words = ['k3', 'k9', 'k3', 'k6', 'k9', 'k7']
rank = {name: i for i, name in enumerate(sorted(set(words)), start=1)}
counts = {}
for name in words:
    counts[name] = counts.get(name, 0) + rank[name]
print(sum(counts.values()))
```

Answer: _____

Q85. Predict the exact single-line output of the following code.

```
matrix = [[10, 6, 10], [3, 7, 3], [6, 10, 7]]
flat = [x for row in matrix for x in row]
unique = sorted(set(flat))
index = {value: idx for idx, value in enumerate(unique)}
print(sum(index[x] for x in flat[:5]))
```

Answer: _____

Q86. Predict the exact single-line output of the following code.

```
words = ['k8', 'k5', 'k8', 'k2', 'k5', 'k2']
rank = {name: i for i, name in enumerate(sorted(set(words)), start=1)}
counts = {}
for name in words:
    counts[name] = counts.get(name, 0) + rank[name]
print(sum(counts.values()))
```

Answer: _____

Q87. Predict the exact single-line output of the following code.

```
matrix = [[11, 12, 11], [1, 9, 1], [12, 11, 9]]
flat = [x for row in matrix for x in row]
unique = sorted(set(flat))
index = {value: idx for idx, value in enumerate(unique)}
print(sum(index[x] for x in flat[:5]))
```

Answer: _____

Q88. Predict the exact single-line output of the following code.

```
matrix = [[8, 5, 8], [6, 4, 6], [5, 8, 4]]
flat = [x for row in matrix for x in row]
unique = sorted(set(flat))
index = {value: idx for idx, value in enumerate(unique)}
print(sum(index[x] for x in flat[:5]))
```

Answer: _____

Q89. Predict the exact single-line output of the following code.

```
pairs = [(3, 3), (3, 2), (3, 9), (3, 9), (2, 9)]
grouped = {}
for k, v in pairs:
    grouped.setdefault(k, []).append(v)
print(sum(len(vs) for vs in grouped.values()) + len(grouped))
```

Answer: _____

Q90. Predict the exact single-line output of the following code.

```
data = [3, 11, 1, 5, 3, 1, 11]
freq = {}
for item in data:
    freq[item] = freq.get(item, 0) + 1
print(sum(key * value for key, value in freq.items()))
```

Answer: _____

Q91. Predict the exact single-line output of the following code.

```
matrix = [[2, 3, 2], [5, 5, 5], [3, 2, 5]]
flat = [x for row in matrix for x in row]
unique = sorted(set(flat))
index = {value: idx for idx, value in enumerate(unique)}
print(sum(index[x] for x in flat[:5]))
```

Answer: _____

Q92. Predict the exact single-line output of the following code.

```
rows = [[9, 3], [3, 2], [12, 5]]
columns = list(zip(*rows))
print(sum(columns[0]) - sum(columns[1]))
```

Answer: _____

Q93. Predict the exact single-line output of the following code.

```
matrix = [[5, 7, 5], [1, 2, 1], [7, 5, 2]]
flat = [x for row in matrix for x in row]
unique = sorted(set(flat))
index = {value: idx for idx, value in enumerate(unique)}
print(sum(index[x] for x in flat[:5]))
```

Answer: _____

Q94. Predict the exact single-line output of the following code.

```
words = ['k3', 'k4', 'k3', 'k4', 'k4', 'k3']
rank = {name: i for i, name in enumerate(sorted(set(words)), start=1)}
counts = {}
for name in words:
    counts[name] = counts.get(name, 0) + rank[name]
print(sum(counts.values()))
```

Answer: _____

Q95. Predict the exact single-line output of the following code.

```
words = ['k5', 'k12', 'k5', 'k6', 'k12', 'k2']
rank = {name: i for i, name in enumerate(sorted(set(words)), start=1)}
counts = {}
for name in words:
    counts[name] = counts.get(name, 0) + rank[name]
print(sum(counts.values()))
```

Answer: _____

Q96. Which statements are correct about Tuple operations in Python? Select all that apply.

- A. Shallow copies duplicate only the outer container and may share nested mutable objects.
- B. Dictionary keys can safely be mutable lists as long as values are immutable.
- C. Tuple hashability depends on all contained elements being hashable.
- D. A single-item tuple requires a trailing comma.

Q97. While applying List vs Tuple vs Array vs Deque (Decision Grid), which statements are technically correct? Select all that apply.

- A. Shallow copies duplicate only the outer container and may share nested mutable objects.
- B. Using the correct collection type often removes the need for complex conditional logic.
- C. Collection choice should be based on access pattern, mutation pattern, and complexity tradeoffs.
- D. Dictionary keys can safely be mutable lists as long as values are immutable.

Q98. Which statements are correct about Returning multiple values in Python? Select all that apply.

- A. Shallow copies duplicate only the outer container and may share nested mutable objects.
- B. List front-pop operations (`pop(0)`) are always $O(1)$, so they are ideal for queue workloads.
- C. Tuple hashability depends on all contained elements being hashable.
- D. Using the correct collection type often removes the need for complex conditional logic.

Q99. In production-quality Python code, which statements about Indexing are valid? Select all that apply.

- A. Tuples allow in-place item reassignment because they are sequence types.
- B. Using the correct collection type often removes the need for complex conditional logic.
- C. A single-item tuple requires a trailing comma.
- D. Collection choice should be based on access pattern, mutation pattern, and complexity tradeoffs.

Q100. While applying Advanced useful classes, which statements are technically correct? Select all that apply.

- A. Shallow copies duplicate only the outer container and may share nested mutable objects.
- B. Dictionary keys can safely be mutable lists as long as values are immutable.
- C. Dictionary view objects (`keys`, `items`) are dynamic and reflect later dictionary updates.
- D. Collection choice should be based on access pattern, mutation pattern, and complexity tradeoffs.

Q101. Predict the exact single-line output of the following code.

```
values = [3, 5, 6, 9, 8, 15, 12]
bucket = {0: [], 1: []}
for value in values:
    bucket[value % 2].append(value)
even = sorted(bucket[0])
odd = sorted(bucket[1], reverse=True)
print((sum(even[:2]) if len(even) >= 2 else sum(even)) + (odd[0] if odd
else 0))
```

Answer: _____

Q102. Predict the exact single-line output of the following code.

```
grid = [[9, 3], [6, 8], [15, 11]]
transposed = list(zip(*grid))
left = list(transposed[0])
right = list(transposed[1])
left.sort(reverse=True)
right.sort()
print(left[0] + right[0] - left[-1])
```

Answer: _____

Q103. Predict the exact single-line output of the following code.

```
values = [6, 9, 2, 3, 15, 5, 9]
bucket = {0: [], 1: []}
for value in values:
    bucket[value % 2].append(value)
even = sorted(bucket[0])
odd = sorted(bucket[1], reverse=True)
print((sum(even[:2]) if len(even) >= 2 else sum(even)) + (odd[0] if odd
else 0))
```

Answer: _____

Q104. Predict the exact single-line output of the following code.

```
grid = [[3, 12], [5, 4], [8, 16]]
transposed = list(zip(*grid))
left = list(transposed[0])
right = list(transposed[1])
left.sort(reverse=True)
right.sort()
print(left[0] + right[0] - left[-1])
```

Answer: _____

Q105. Predict the exact single-line output of the following code.

```
values = [2, 3, 4, 7, 5, 11, 9]
bucket = {0: [], 1: []}
for value in values:
    bucket[value % 2].append(value)
even = sorted(bucket[0])
odd = sorted(bucket[1], reverse=True)
print((sum(even[:2]) if len(even) >= 2 else sum(even)) + (odd[0] if odd
else 0))
```

Answer: _____

Q106. Predict the exact single-line output of the following code.

```
values = [9, 9, 2, 4, 18, 6, 13]
bucket = {0: [], 1: []}
for value in values:
    bucket[value % 2].append(value)
even = sorted(bucket[0])
odd = sorted(bucket[1], reverse=True)
print((sum(even[:2]) if len(even) >= 2 else sum(even)) + (odd[0] if odd
else 0))
```

Answer: _____

Q107. Predict the exact single-line output of the following code.

```
base = {'u': [10, 11, 6], 'v': [11, 6, 7], 'w': [10, 7]}
flattened = [(k, value) for k, values in base.items() for value in values]
weights = {'u': 2, 'v': 3, 'w': 5}
print(sum(value * weights[key] for key, value in flattened if value % 2
== 0))
```

Answer: _____

Q108. Predict the exact single-line output of the following code.

```
pairs = [(11, 7), (7, 4), (11, 4), (4, 4), (7, 4), (11, 4)]
adj = {}
for x, y in pairs:
    adj.setdefault(x, set()).add(y)
    adj.setdefault(y, set()).add(x)
print(sum(len(v) for v in adj.values()))
```

Answer: _____

Q109. Predict the exact single-line output of the following code.

```
grid = [[7, 5], [1, 3], [8, 8]]
transposed = list(zip(*grid))
left = list(transposed[0])
right = list(transposed[1])
left.sort(reverse=True)
right.sort()
print(left[0] + right[0] - left[-1])
```

Answer: _____

Q110. Predict the exact single-line output of the following code.

```
grid = [[7, 9], [5, 5], [12, 14]]
transposed = list(zip(*grid))
left = list(transposed[0])
right = list(transposed[1])
left.sort(reverse=True)
right.sort()
print(left[0] + right[0] - left[-1])
```

Answer: _____

Q111. Predict the exact single-line output of the following code.

```
values = [3, 9, 5, 4, 12, 9, 7]
bucket = {0: [], 1: []}
for value in values:
    bucket[value % 2].append(value)
even = sorted(bucket[0])
odd = sorted(bucket[1], reverse=True)
print((sum(even[:2]) if len(even) >= 2 else sum(even)) + (odd[0] if odd
else 0))
```

Answer: _____

Q112. Predict the exact single-line output of the following code.

```
pairs = [(10, 4), (4, 4), (10, 4), (4, 4), (4, 4), (10, 4)]
adj = {}
for x, y in pairs:
    adj.setdefault(x, set()).add(y)
    adj.setdefault(y, set()).add(x)
print(sum(len(v) for v in adj.values()))
```

Answer: _____

Q113. Predict the exact single-line output of the following code.

```
rows = [[11, 8, 2], [2, 9, 11], [8, 11, 9]]
diag = [rows[i][i] for i in range(3)]
anti = [rows[i][2 - i] for i in range(3)]
freq = {}
for x in diag + anti:
    freq[x] = freq.get(x, 0) + 1
print(sum(v * k for k, v in freq.items()))
```

Answer: _____

Q114. Predict the exact single-line output of the following code.

```
grid = [[5, 9], [7, 9], [12, 18]]
transposed = list(zip(*grid))
left = list(transposed[0])
right = list(transposed[1])
left.sort(reverse=True)
right.sort()
print(left[0] + right[0] - left[-1])
```

Answer: _____

Q115. Predict the exact single-line output of the following code.

```
rows = [[3, 7, 6], [6, 9, 3], [7, 3, 9]]
diag = [rows[i][i] for i in range(3)]
anti = [rows[i][2 - i] for i in range(3)]
freq = {}
for x in diag + anti:
    freq[x] = freq.get(x, 0) + 1
print(sum(v * k for k, v in freq.items()))
```

Answer: _____

Q116. Predict the exact single-line output of the following code.

```
rows = [[6, 3, 7], [7, 8, 6], [3, 6, 8]]
diag = [rows[i][i] for i in range(3)]
anti = [rows[i][2 - i] for i in range(3)]
freq = {}
for x in diag + anti:
    freq[x] = freq.get(x, 0) + 1
print(sum(v * k for k, v in freq.items()))
```

Answer: _____

Q117. Predict the exact single-line output of the following code.

```
grid = [[11, 9], [1, 7], [12, 16]]
transposed = list(zip(*grid))
left = list(transposed[0])
right = list(transposed[1])
left.sort(reverse=True)
right.sort()
print(left[0] + right[0] - left[-1])
```

Answer: _____

Q118. Predict the exact single-line output of the following code.

```
rows = [[5, 12, 2], [2, 2, 5], [12, 5, 2]]
diag = [rows[i][i] for i in range(3)]
anti = [rows[i][2 - i] for i in range(3)]
freq = {}
for x in diag + anti:
    freq[x] = freq.get(x, 0) + 1
print(sum(v * k for k, v in freq.items()))
```

Answer: _____

Q119. Predict the exact single-line output of the following code.

```
values = [6, 11, 3, 2, 17, 5, 8]
bucket = {0: [], 1: []}
for value in values:
    bucket[value % 2].append(value)
even = sorted(bucket[0])
odd = sorted(bucket[1], reverse=True)
print((sum(even[:2]) if len(even) >= 2 else sum(even)) + (odd[0] if odd
else 0))
```

Answer: _____

Q120. Predict the exact single-line output of the following code.

```
rows = [[2, 5, 7], [7, 5, 2], [5, 2, 5]]
diag = [rows[i][i] for i in range(3)]
anti = [rows[i][2 - i] for i in range(3)]
freq = {}
for x in diag + anti:
    freq[x] = freq.get(x, 0) + 1
print(sum(v * k for k, v in freq.items()))
```

Answer: _____

Q121. Predict the exact single-line output of the following code.

```
rows = [[6, 12, 2], [2, 9, 6], [12, 6, 9]]
diag = [rows[i][i] for i in range(3)]
anti = [rows[i][2 - i] for i in range(3)]
freq = {}
for x in diag + anti:
    freq[x] = freq.get(x, 0) + 1
print(sum(v * k for k, v in freq.items()))
```

Answer: _____

Q122. Predict the exact single-line output of the following code.

```
rows = [[4, 5, 2], [2, 6, 4], [5, 4, 6]]
diag = [rows[i][i] for i in range(3)]
anti = [rows[i][2 - i] for i in range(3)]
freq = {}
for x in diag + anti:
    freq[x] = freq.get(x, 0) + 1
print(sum(v * k for k, v in freq.items()))
```

Answer: _____

Q123. Predict the exact single-line output of the following code.

```
rows = [[6, 6, 5], [5, 8, 6], [6, 6, 8]]
diag = [rows[i][i] for i in range(3)]
anti = [rows[i][2 - i] for i in range(3)]
freq = {}
for x in diag + anti:
    freq[x] = freq.get(x, 0) + 1
print(sum(v * k for k, v in freq.items()))
```

Answer: _____

Q124. Predict the exact single-line output of the following code.

```
pairs = [(3, 8), (8, 1), (3, 6), (1, 6), (8, 6), (3, 1)]
adj = {}
for x, y in pairs:
    adj.setdefault(x, set()).add(y)
    adj.setdefault(y, set()).add(x)
print(sum(len(v) for v in adj.values()))
```

Answer: _____

Q125. Predict the exact single-line output of the following code.

```
base = {'u': [7, 5, 7], 'v': [5, 7, 2], 'w': [7, 2]}
flattened = [(k, value) for k, values in base.items() for value in values]
weights = {'u': 2, 'v': 3, 'w': 5}
print(sum(value * weights[key] for key, value in flattened if value % 2
== 0))
```

Answer: _____

Q126. Which statements are correct about Task in Python? Select all that apply.

- A. Async code automatically speeds up CPU-bound loops without offloading or multiprocessing.
- B. In asyncio, cancellation is cooperative and usually observed at await points.
- C. Correctness and safe shutdown must be verified before performance tuning claims.
- D. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.

Q127. In production-quality Python code, which statements about Debugging Stuck Threaded Programs are valid? Select all that apply.

- A. Observability (task/thread ids, queue depth, retries) is critical for production debugging.
- B. Async code automatically speeds up CPU-bound loops without offloading or multiprocessing.
- C. In asyncio, cancellation is cooperative and usually observed at await points.
- D. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.

Q128. In production-quality Python code, which statements about Common Production Pitfalls are valid? Select all that apply.

- A. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.
- B. Using `asyncio.gather` makes timeout handling unnecessary.
- C. Correctness and safe shutdown must be verified before performance tuning claims.
- D. Blocking calls inside the event loop should be offloaded or replaced with async APIs.

Q129. Which statements are correct about submit vs map in Python? Select all that apply.

- A. If a thread is already running, `future.cancel()` always stops it immediately.
- B. Using many threads removes the need for lock or shared-state design.
- C. Concurrency design should include bounded work queues or bounded in-flight operations.
- D. Correctness and safe shutdown must be verified before performance tuning claims.

Q130. Predict the exact single-line output of the following code.

```
from concurrent.futures import ThreadPoolExecutor
def transform(value):
    return value * 2
with ThreadPoolExecutor(max_workers=2) as pool:
    results = list(pool.map(transform, [2, 6, 3]))
print(sum(results))
```

Answer: _____

Q131. Predict the exact single-line output of the following code.

```
import asyncio
async def worker(value, offset):
    return value + offset
async def main():
    results = await asyncio.gather(worker(9, 6), worker(5, 6))
    return sum(results)
print(asyncio.run(main()))
```

Answer: _____

Q132. Predict the exact single-line output of the following code.

```
import threading
shared_total = 0
lock = threading.Lock()
def add(value):
    global shared_total
    with lock:
        shared_total += value
for value in [6, 12, 2]:
    add(value)
print(shared_total)
```

Answer: _____

Q133. While applying Team/Codebase Constraints, which statements are technically correct? Select all that apply.

- A. Deadlocks can happen only when code uses more than two locks.
- B. Concurrency design should include explicit timeout, cancellation, and failure behavior.
- C. Blocking calls inside the event loop should be offloaded or replaced with async APIs.
- D. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.

Q134. While applying Why Use Threads in Python, which statements are technically correct? Select all that apply.

- A. Concurrency design should include explicit timeout, cancellation, and failure behavior.
- B. Deadlocks can happen only when code uses more than two locks.
- C. Blocking calls inside the event loop should be offloaded or replaced with async APIs.
- D. Using many threads removes the need for lock or shared-state design.

Q135. In production-quality Python code, which statements about Fundamentals are valid? Select all that apply.

- A. If a thread is already running, `future.cancel()` always stops it immediately.
- B. Using `asyncio.gather` makes timeout handling unnecessary.
- C. Observability (task/thread ids, queue depth, retries) is critical for production debugging.
- D. Blocking calls inside the event loop should be offloaded or replaced with async APIs.

Q136. In production-quality Python code, which statements about Why is `time.sleep` bad in async code? are valid? Select all that apply.

- A. Correctness and safe shutdown must be verified before performance tuning claims.
- B. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.
- C. In asyncio, cancellation is cooperative and usually observed at await points.
- D. Using `asyncio.gather` makes timeout handling unnecessary.

Q137. While applying Performance Measurement Basics, which statements are technically correct? Select all that apply.

- A. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.
- B. Correctness and safe shutdown must be verified before performance tuning claims.
- C. Deadlocks can happen only when code uses more than two locks.
- D. Observability (task/thread ids, queue depth, retries) is critical for production debugging.

Q138. While applying Error Handling in Sync Code, which statements are technically correct? Select all that apply.

- A. If a thread is already running, `future.cancel()` always stops it immediately.
- B. Concurrency design should include explicit timeout, cancellation, and failure behavior.
- C. Blocking calls inside the event loop should be offloaded or replaced with async APIs.
- D. Using many threads removes the need for lock or shared-state design.

Q139. Which statements are correct about Practice Assignment in Python? Select all that apply.

- A. Blocking calls inside the event loop should be offloaded or replaced with async APIs.
- B. Correctness and safe shutdown must be verified before performance tuning claims.
- C. If a thread is already running, `future.cancel()` always stops it immediately.
- D. Deadlocks can happen only when code uses more than two locks.

Q140. In production-quality Python code, which statements about One-Page Summary are valid? Select all that apply.

- A. If a thread is already running, `future.cancel()` always stops it immediately.
- B. In asyncio, cancellation is cooperative and usually observed at await points.
- C. Using many threads removes the need for lock or shared-state design.
- D. Observability (task/thread ids, queue depth, retries) is critical for production debugging.

Q141. While applying Rapid Fire (One-Liners), which statements are technically correct? Select all that apply.

- A. Concurrency design should include bounded work queues or bounded in-flight operations.
- B. If a thread is already running, `future.cancel()` always stops it immediately.
- C. Using many threads removes the need for lock or shared-state design.
- D. Correctness and safe shutdown must be verified before performance tuning claims.

Q142. In production-quality Python code, which statements about Async are valid? Select all that apply.

- A. Observability (task/thread ids, queue depth, retries) is critical for production debugging.
- B. Async code automatically speeds up CPU-bound loops without offloading or multiprocessing.
- C. Concurrency design should include explicit timeout, cancellation, and failure behavior.
- D. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.

Q143. In production-quality Python code, which statements about Debugging Async are valid? Select all that apply.

- A. Observability (task/thread ids, queue depth, retries) is critical for production debugging.
- B. Concurrency design should include explicit timeout, cancellation, and failure behavior.
- C. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.
- D. If a thread is already running, `future.cancel()` always stops it immediately.

Q144. Which statements are correct about Async Locks for Shared Mutable State in Python? Select all that apply.

- A. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.
- B. Deadlocks can happen only when code uses more than two locks.
- C. Blocking calls inside the event loop should be offloaded or replaced with async APIs.
- D. Observability (task/thread ids, queue depth, retries) is critical for production debugging.

Q145. While applying What Is Multithreading, which statements are technically correct? Select all that apply.

- A. Blocking calls inside the event loop should be offloaded or replaced with async APIs.
- B. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.
- C. Async code automatically speeds up CPU-bound loops without offloading or multiprocessing.
- D. In asyncio, cancellation is cooperative and usually observed at await points.

Q146. While applying How to prevent race conditions?, which statements are technically correct? Select all that apply.

- A. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.
- B. Using `asyncio.gather` makes timeout handling unnecessary.
- C. In asyncio, cancellation is cooperative and usually observed at await points.
- D. Observability (task/thread ids, queue depth, retries) is critical for production debugging.

Q147. In production-quality Python code, which statements about Why This Comparison Is Asked Often are valid? Select all that apply.

- A. Blocking calls inside the event loop should be offloaded or replaced with async APIs.
- B. Deadlocks can happen only when code uses more than two locks.
- C. Concurrency design should include explicit timeout, cancellation, and failure behavior.
- D. Using ``asyncio.gather`` makes timeout handling unnecessary.

Q148. While applying CPU-bound vs I/O-bound in Sync Design, which statements are technically correct? Select all that apply.

- A. If a thread is already running, ``future.cancel()`` always stops it immediately.
- B. Observability (task/thread ids, queue depth, retries) is critical for production debugging.
- C. Using many threads removes the need for lock or shared-state design.
- D. Concurrency design should include bounded work queues or bounded in-flight operations.

Q149. While applying Timeouts and Retries in Sync Systems, which statements are technically correct? Select all that apply.

- A. Observability (task/thread ids, queue depth, retries) is critical for production debugging.
- B. Deadlocks can happen only when code uses more than two locks.
- C. Blocking calls inside the event loop should be offloaded or replaced with async APIs.
- D. Using many threads removes the need for lock or shared-state design.

Q150. In production-quality Python code, which statements about What is event loop? are valid? Select all that apply.

- A. Deadlocks can happen only when code uses more than two locks.
- B. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.
- C. Concurrency design should include explicit timeout, cancellation, and failure behavior.
- D. Observability (task/thread ids, queue depth, retries) is critical for production debugging.

Q151. While applying Environment Setup, which statements are technically correct? Select all that apply.

- A. Blocking calls inside the event loop should be offloaded or replaced with async APIs.
- B. Concurrency design should include explicit timeout, cancellation, and failure behavior.
- C. Using many threads removes the need for lock or shared-state design.
- D. Async code automatically speeds up CPU-bound loops without offloading or multiprocessing.

Q152. In production-quality Python code, which statements about Common Threading Mistakes are valid? Select all that apply.

- A. Blocking calls inside the event loop should be offloaded or replaced with async APIs.
- B. If a thread is already running, ``future.cancel()`` always stops it immediately.
- C. Deadlocks can happen only when code uses more than two locks.
- D. Concurrency design should include explicit timeout, cancellation, and failure behavior.

Q153. Which statements are correct about Decision Matrix in Python? Select all that apply.

- A. If a thread is already running, ``future.cancel()`` always stops it immediately.
- B. Using many threads removes the need for lock or shared-state design.
- C. Concurrency design should include explicit timeout, cancellation, and failure behavior.
- D. Blocking calls inside the event loop should be offloaded or replaced with async APIs.

Q154. In production-quality Python code, which statements about RLock are valid? Select all that apply.

- A. Using ``asyncio.gather`` makes timeout handling unnecessary.
- B. In `asyncio`, cancellation is cooperative and usually observed at await points.
- C. Concurrency design should include explicit timeout, cancellation, and failure behavior.
- D. Deadlocks can happen only when code uses more than two locks.

Q155. Predict the exact single-line output of the following code.

```
import queue
buffer = queue.Queue()
for item in [10, 10, 2]:
    buffer.put(item)
first = buffer.get()
second = buffer.get()
print(first * second)
```

Answer: _____

Q156. Predict the exact single-line output of the following code.

```
import threading
shared_total = 0
lock = threading.Lock()
def add(value):
    global shared_total
    with lock:
        shared_total += value
for value in [9, 4, 4]:
    add(value)
print(shared_total)
```

Answer: _____

Q157. Predict the exact single-line output of the following code.

```
import threading
shared_total = 0
lock = threading.Lock()
def add(value):
    global shared_total
    with lock:
        shared_total += value
for value in [5, 7, 5]:
    add(value)
print(shared_total)
```

Answer: _____

Q158. Predict the exact single-line output of the following code.

```
import asyncio
async def worker(value, offset):
    return value + offset
async def main():
    results = await asyncio.gather(worker(7, 3), worker(4, 3))
    return sum(results)
print(asyncio.run(main()))
```

Answer: _____

Q159. Predict the exact single-line output of the following code.

```
def stage(value, bias):  
    return value + bias  
left = stage(10, 2)  
right = stage(10, 2)  
print(abs(left - right))
```

Answer: _____

Q160. Predict the exact single-line output of the following code.

```
import queue  
buffer = queue.Queue()  
for item in [3, 8, 3]:  
    buffer.put(item)  
first = buffer.get()  
second = buffer.get()  
print(first * second)
```

Answer: _____

Q161. Predict the exact single-line output of the following code.

```
import threading  
shared_total = 0  
lock = threading.Lock()  
def add(value):  
    global shared_total  
    with lock:  
        shared_total += value  
for value in [3, 6, 6]:  
    add(value)  
print(shared_total)
```

Answer: _____

Q162. Predict the exact single-line output of the following code.

```
def stage(value, bias):  
    return value + bias  
left = stage(7, 9)  
right = stage(9, 9)  
print(abs(left - right))
```

Answer: _____

Q163. Predict the exact single-line output of the following code.

```
def stage(value, bias):  
    return value + bias  
left = stage(3, 9)  
right = stage(3, 9)  
print(abs(left - right))
```

Answer: _____

Q164. Predict the exact single-line output of the following code.

```
import threading
shared_total = 0
lock = threading.Lock()
def add(value):
    global shared_total
    with lock:
        shared_total += value
for value in [10, 3, 3]:
    add(value)
print(shared_total)
```

Answer: _____

Q165. Predict the exact single-line output of the following code.

```
import queue
buffer = queue.Queue()
for item in [7, 5, 2]:
    buffer.put(item)
first = buffer.get()
second = buffer.get()
print(first * second)
```

Answer: _____

Q166. Predict the exact single-line output of the following code.

```
def stage(value, bias):
    return value + bias
left = stage(4, 2)
right = stage(3, 2)
print(abs(left - right))
```

Answer: _____

Q167. Predict the exact single-line output of the following code.

```
import threading
shared_total = 0
lock = threading.Lock()
def add(value):
    global shared_total
    with lock:
        shared_total += value
for value in [6, 3, 4]:
    add(value)
print(shared_total)
```

Answer: _____

Q168. Predict the exact single-line output of the following code.

```
import threading
shared_total = 0
lock = threading.Lock()
def add(value):
    global shared_total
    with lock:
        shared_total += value
for value in [2, 6, 5]:
    add(value)
print(shared_total)
```

Answer: _____

Q169. Predict the exact single-line output of the following code.

```
from concurrent.futures import ThreadPoolExecutor
def transform(value):
    return value * 2
with ThreadPoolExecutor(max_workers=2) as pool:
    results = list(pool.map(transform, [5, 10, 2]))
print(sum(results))
```

Answer: _____

Q170. Predict the exact single-line output of the following code.

```
from concurrent.futures import ThreadPoolExecutor
def transform(value):
    return value * 2
with ThreadPoolExecutor(max_workers=2) as pool:
    results = list(pool.map(transform, [6, 5, 3]))
print(sum(results))
```

Answer: _____

Q171. Predict the exact single-line output of the following code.

```
import queue
buffer = queue.Queue()
for item in [10, 7, 2]:
    buffer.put(item)
first = buffer.get()
second = buffer.get()
print(first * second)
```

Answer: _____

Q172. Predict the exact single-line output of the following code.

```
from concurrent.futures import ThreadPoolExecutor
def transform(value):
    return value * 2
with ThreadPoolExecutor(max_workers=2) as pool:
    results = list(pool.map(transform, [3, 6, 1]))
print(sum(results))
```

Answer: _____

Q173. Predict the exact single-line output of the following code.

```
from concurrent.futures import ThreadPoolExecutor
def transform(value):
    return value * 2
with ThreadPoolExecutor(max_workers=2) as pool:
    results = list(pool.map(transform, [11, 3, 4]))
print(sum(results))
```

Answer: _____

Q174. Predict the exact single-line output of the following code.

```
import threading
shared_total = 0
lock = threading.Lock()
def add(value):
    global shared_total
    with lock:
        shared_total += value
for value in [6, 7, 5]:
    add(value)
print(shared_total)
```

Answer: _____

Q175. Predict the exact single-line output of the following code.

```
import threading
shared_total = 0
lock = threading.Lock()
def add(value):
    global shared_total
    with lock:
        shared_total += value
for value in [5, 8, 4]:
    add(value)
print(shared_total)
```

Answer: _____

Q176. Predict the exact single-line output of the following code.

```
import queue
buffer = queue.Queue()
for item in [9, 9, 6]:
    buffer.put(item)
first = buffer.get()
second = buffer.get()
print(first * second)
```

Answer: _____

Q177. Predict the exact single-line output of the following code.

```
from concurrent.futures import ThreadPoolExecutor
def transform(value):
    return value * 2
with ThreadPoolExecutor(max_workers=2) as pool:
    results = list(pool.map(transform, [5, 9, 6]))
print(sum(results))
```

Answer: _____

Q178. While applying Condition, which statements are technically correct? Select all that apply.

- A. Using ``asyncio.gather`` makes timeout handling unnecessary.
- B. Concurrency design should include bounded work queues or bounded in-flight operations.
- C. If a thread is already running, ``future.cancel()`` always stops it immediately.
- D. Concurrency design should include explicit timeout, cancellation, and failure behavior.

Q179. Which statements are correct about Correctness Needs in Python? Select all that apply.

- A. Concurrency design should include explicit timeout, cancellation, and failure behavior.
- B. In `asyncio`, cancellation is cooperative and usually observed at await points.
- C. If a thread is already running, ``future.cancel()`` always stops it immediately.
- D. Deadlocks can happen only when code uses more than two locks.

Q180. While applying Structured Concurrency With TaskGroup (Python 3.11+), which statements are technically correct? Select all that apply.

- A. If a thread is already running, ``future.cancel()`` always stops it immediately.
- B. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.
- C. Correctness and safe shutdown must be verified before performance tuning claims.
- D. Blocking calls inside the event loop should be offloaded or replaced with `async` APIs.

Q181. While applying Retry Policy in Async Systems, which statements are technically correct? Select all that apply.

- A. Observability (task/thread ids, queue depth, retries) is critical for production debugging.
- B. Using ``asyncio.gather`` makes timeout handling unnecessary.
- C. Concurrency design should include explicit timeout, cancellation, and failure behavior.
- D. Using many threads removes the need for lock or shared-state design.

Q182. Which statements are correct about What is backpressure? in Python? Select all that apply.

- A. Observability (task/thread ids, queue depth, retries) is critical for production debugging.
- B. If a thread is already running, ``future.cancel()`` always stops it immediately.
- C. Blocking calls inside the event loop should be offloaded or replaced with `async` APIs.
- D. `Async` code automatically speeds up CPU-bound loops without offloading or multiprocessing.

Q183. While applying Practice Assignment, which statements are technically correct? Select all that apply.

- A. Blocking calls inside the event loop should be offloaded or replaced with `async` APIs.
- B. Concurrency design should include explicit timeout, cancellation, and failure behavior.
- C. `Async` code automatically speeds up CPU-bound loops without offloading or multiprocessing.
- D. Using ``asyncio.gather`` makes timeout handling unnecessary.

Q184. While applying Common red flags in `async` code review?, which statements are technically correct? Select all that apply.

- A. Concurrency design should include bounded work queues or bounded in-flight operations.
- B. Deadlocks can happen only when code uses more than two locks.
- C. Observability (task/thread ids, queue depth, retries) is critical for production debugging.
- D. Using ``asyncio.gather`` makes timeout handling unnecessary.

Q185. Predict the exact single-line output of the following code.

```
import asyncio
async def worker(value, offset):
    return value + offset
async def main():
    results = await asyncio.gather(worker(2, 5), worker(10, 5))
    return sum(results)
print(asyncio.run(main()))
```

Answer: _____

Q186. Predict the exact single-line output of the following code.

```
import threading
shared_total = 0
lock = threading.Lock()
def add(value):
    global shared_total
    with lock:
        shared_total += value
for value in [7, 12, 5]:
    add(value)
print(shared_total)
```

Answer: _____

Q187. Predict the exact single-line output of the following code.

```
import asyncio
async def worker(value, offset):
    return value + offset
async def main():
    results = await asyncio.gather(worker(11, 6), worker(12, 6))
    return sum(results)
print(asyncio.run(main()))
```

Answer: _____

Q188. Predict the exact single-line output of the following code.

```
import queue
buffer = queue.Queue()
for item in [7, 9, 6]:
    buffer.put(item)
first = buffer.get()
second = buffer.get()
print(first * second)
```

Answer: _____

Q189. Predict the exact single-line output of the following code.

```
import threading
shared_total = 0
lock = threading.Lock()
def add(value):
    global shared_total
    with lock:
        shared_total += value
for value in [4, 8, 6]:
    add(value)
print(shared_total)
```

Answer: _____

Q190. Predict the exact single-line output of the following code.

```
def stage(value, bias):
    return value + bias
left = stage(9, 6)
right = stage(12, 6)
print(abs(left - right))
```

Answer: _____

Q191. Predict the exact single-line output of the following code.

```
from concurrent.futures import ThreadPoolExecutor
def transform(value):
    return value * 2
with ThreadPoolExecutor(max_workers=2) as pool:
    results = list(pool.map(transform, [2, 6, 1]))
print(sum(results))
```

Answer: _____

Q192. Predict the exact single-line output of the following code.

```
import threading
shared_total = 0
lock = threading.Lock()
def add(value):
    global shared_total
    with lock:
        shared_total += value
for value in [9, 11, 4]:
    add(value)
print(shared_total)
```

Answer: _____

Q193. In production-quality Python code, which statements about What is the difference between concurrency and parallelism? are valid? Select all that apply.

- A. Correctness and safe shutdown must be verified before performance tuning claims.
- B. Blocking calls inside the event loop should be offloaded or replaced with async APIs.
- C. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.
- D. In asyncio, cancellation is cooperative and usually observed at await points.

Q194. While applying Bounded Resource Access With Semaphore, which statements are technically correct? Select all that apply.

- A. Concurrency design should include bounded work queues or bounded in-flight operations.
- B. Blocking calls inside the event loop should be offloaded or replaced with async APIs.
- C. Correctness and safe shutdown must be verified before performance tuning claims.
- D. Awaiting a coroutine blocks all other tasks until the coroutine fully completes.

Q195. While applying What is a blocking call?, which statements are technically correct? Select all that apply.

- A. Observability (task/thread ids, queue depth, retries) is critical for production debugging.
- B. If a thread is already running, `future.cancel()` always stops it immediately.
- C. Blocking calls inside the event loop should be offloaded or replaced with async APIs.
- D. Concurrency design should include explicit timeout, cancellation, and failure behavior.

Q196. While applying CPU-bound, which statements are technically correct? Select all that apply.

- A. If a thread is already running, `future.cancel()` always stops it immediately.
- B. Observability (task/thread ids, queue depth, retries) is critical for production debugging.
- C. In asyncio, cancellation is cooperative and usually observed at await points.
- D. Concurrency design should include explicit timeout, cancellation, and failure behavior.

Q197. Predict the exact single-line output of the following code.

```
import queue
buffer = queue.Queue()
for item in [8, 12, 7]:
    buffer.put(item)
first = buffer.get()
second = buffer.get()
print(first * second)
```

Answer: _____

Q198. Predict the exact single-line output of the following code.

```
def stage(value, bias):
    return value + bias
left = stage(3, 8)
right = stage(8, 8)
print(abs(left - right))
```

Answer: _____

Q199. Predict the exact single-line output of the following code.

```
import asyncio
async def worker(value, offset):
    return value + offset
async def main():
    results = await asyncio.gather(worker(9, 6), worker(6, 6))
    return sum(results)
print(asyncio.run(main()))
```

Answer: _____

Q200. Predict the exact single-line output of the following code.

```
def stage(value, bias):  
    return value + bias  
left = stage(11, 8)  
right = stage(8, 8)  
print(abs(left - right))
```

Answer: _____

Q201. Which statements are correct about Refactor interview question in Python? Select all that apply.

- A. Protocols require explicit inheritance from the protocol class at runtime.
- B. Depending on abstractions improves testability and implementation swapping.
- C. Composition is often safer than deep inheritance for change-heavy systems.
- D. Dependency injection is mainly about reducing lines of code, not testability.

Q202. While applying When to use, which statements are technically correct? Select all that apply.

- A. Factory-style creation helps isolate construction logic from business workflows.
- B. OOP design should preserve clear contracts and minimize unnecessary coupling.
- C. Protocols require explicit inheritance from the protocol class at runtime.
- D. Inheritance automatically guarantees low coupling and high cohesion.

Q203. Which statements are correct about Mixins vs Normal Base Classes in Python? Select all that apply.

- A. Inheritance automatically guarantees low coupling and high cohesion.
- B. Substitutability requires preserving semantic expectations, not just method names.
- C. Overriding a method should always change input/output contract details.
- D. Depending on abstractions improves testability and implementation swapping.

Q204. Predict the exact single-line output of the following code.

```
class Base:  
    def compute(self): return 5  
class Child(Base):  
    def compute(self): return super().compute() + 5  
print(Child().compute())
```

Answer: _____

Q205. Predict the exact single-line output of the following code.

```
class Engine:  
    def power(self): return 10  
class Car:  
    def __init__(self, engine):  
        self.engine = engine  
    def score(self):  
        return self.engine.power() + 5  
print(Car(Engine()).score())
```

Answer: _____

Q206. Predict the exact single-line output of the following code.

```
class S:  
    def __init__(self): self.n = 9  
    def __repr__(self):  
        return f'S(value={self.n})'  
print(S())
```

Answer: _____

Q207. Predict the exact single-line output of the following code.

```
class Engine:
    def power(self): return 8
class Car:
    def __init__(self, engine):
        self.engine = engine
    def score(self):
        return self.engine.power() + 5
print(Car(Engine()).score())
```

Answer: _____

Q208. Which statements are correct about Class Attributes vs Instance Attributes in Python? Select all that apply.

- A. Polymorphism always requires explicit inheritance from a common base class.
- B. Protocols require explicit inheritance from the protocol class at runtime.
- C. Polymorphism allows extending behavior without rewriting stable caller logic.
- D. Depending on abstractions improves testability and implementation swapping.

Q209. While applying Public, Protected and Private in Python, which statements are technically correct? Select all that apply.

- A. Encapsulation protects invariants by exposing behavior methods instead of raw state mutation.
- B. Composition is often safer than deep inheritance for change-heavy systems.
- C. Dependency injection is mainly about reducing lines of code, not testability.
- D. Polymorphism always requires explicit inheritance from a common base class.

Q210. Which statements are correct about Refactor interview question in Python? Select all that apply.

- A. LSP is satisfied as long as child method names match parent method names.
- B. Dependency injection is mainly about reducing lines of code, not testability.
- C. Depending on abstractions improves testability and implementation swapping.
- D. Substitutability requires preserving semantic expectations, not just method names.

Q211. Which statements are correct about Common SOLID Violations to Spot in Review in Python? Select all that apply.

- A. Substitutability requires preserving semantic expectations, not just method names.
- B. Composition is often safer than deep inheritance for change-heavy systems.
- C. LSP is satisfied as long as child method names match parent method names.
- D. ISP recommends one large interface so every client shares identical methods.

Q212. While applying Contract Design for Safe Polymorphism, which statements are technically correct? Select all that apply.

- A. LSP is satisfied as long as child method names match parent method names.
- B. Polymorphism allows extending behavior without rewriting stable caller logic.
- C. Encapsulation protects invariants by exposing behavior methods instead of raw state mutation.
- D. Polymorphism always requires explicit inheritance from a common base class.

Q213. In production-quality Python code, which statements about self are valid? Select all that apply.

- A. OOP design should preserve clear contracts and minimize unnecessary coupling.
- B. Protocols require explicit inheritance from the protocol class at runtime.
- C. Depending on abstractions improves testability and implementation swapping.
- D. LSP is satisfied as long as child method names match parent method names.

Q214. Which statements are correct about Mini Project Pattern: Inventory Product Class in Python? Select all that apply.

- A. Depending on abstractions improves testability and implementation swapping.
- B. Encapsulation protects invariants by exposing behavior methods instead of raw state mutation.
- C. Inheritance automatically guarantees low coupling and high cohesion.
- D. Polymorphism always requires explicit inheritance from a common base class.

Q215. While applying Real Example: Employee Class, which statements are technically correct? Select all that apply.

- A. Polymorphism always requires explicit inheritance from a common base class.
- B. Composition is often safer than deep inheritance for change-heavy systems.
- C. Substitutability requires preserving semantic expectations, not just method names.
- D. LSP is satisfied as long as child method names match parent method names.

Q216. Which statements are correct about Factory Method Pattern in Python? Select all that apply.

- A. OOP design should preserve clear contracts and minimize unnecessary coupling.
- B. Dependency injection is mainly about reducing lines of code, not testability.
- C. Composition is often safer than deep inheritance for change-heavy systems.
- D. Inheritance automatically guarantees low coupling and high cohesion.

Q217. Which statements are correct about When Not to Use Polymorphism in Python? Select all that apply.

- A. Polymorphism allows extending behavior without rewriting stable caller logic.
- B. Protocols require explicit inheritance from the protocol class at runtime.
- C. Encapsulation protects invariants by exposing behavior methods instead of raw state mutation.
- D. Dependency injection is mainly about reducing lines of code, not testability.

Q218. In production-quality Python code, which statements about Encapsulation are valid? Select all that apply.

- A. Substitutability requires preserving semantic expectations, not just method names.
- B. Protocols require explicit inheritance from the protocol class at runtime.
- C. Composition is often safer than deep inheritance for change-heavy systems.
- D. Polymorphism always requires explicit inheritance from a common base class.

Q219. In production-quality Python code, which statements about When to Create a Class (Decision Framework) are valid? Select all that apply.

- A. Dependency injection is mainly about reducing lines of code, not testability.
- B. Polymorphism allows extending behavior without rewriting stable caller logic.
- C. Composition is often safer than deep inheritance for change-heavy systems.
- D. Polymorphism always requires explicit inheritance from a common base class.

Q220. While applying Way E: Decorator Registration Style, which statements are technically correct? Select all that apply.

- A. Depending on abstractions improves testability and implementation swapping.
- B. Inheritance automatically guarantees low coupling and high cohesion.
- C. Protocols require explicit inheritance from the protocol class at runtime.
- D. Substitutability requires preserving semantic expectations, not just method names.

Q221. Which statements are correct about Why Inheritance Exists in Python? Select all that apply.

- A. Inheritance automatically guarantees low coupling and high cohesion.
- B. Cooperative multiple inheritance relies on consistent `super()` usage across classes.
- C. Overriding a method should always change input/output contract details.
- D. Substitutability requires preserving semantic expectations, not just method names.

Q222. In production-quality Python code, which statements about Static method are valid? Select all that apply.

- A. Protocols require explicit inheritance from the protocol class at runtime.
- B. Polymorphism allows extending behavior without rewriting stable caller logic.
- C. LSP is satisfied as long as child method names match parent method names.
- D. Depending on abstractions improves testability and implementation swapping.

Q223. Which statements are correct about Protected and Private in Inheritance in Python? Select all that apply.

- A. Composition is often safer than deep inheritance for change-heavy systems.
- B. Protocols require explicit inheritance from the protocol class at runtime.
- C. Deep inheritance trees can increase fragility when base classes change frequently.
- D. Inheritance automatically guarantees low coupling and high cohesion.

Q224. In production-quality Python code, which statements about Python snippet are valid? Select all that apply.

- A. LSP is satisfied as long as child method names match parent method names.
- B. Depending on abstractions improves testability and implementation swapping.
- C. Factory-style creation helps isolate construction logic from business workflows.
- D. Strategy pattern requires inheritance and cannot use composition.

Q225. In production-quality Python code, which statements about Where to use are valid? Select all that apply.

- A. Dependency injection is mainly about reducing lines of code, not testability.
- B. Substitutability requires preserving semantic expectations, not just method names.
- C. Strategy pattern requires inheritance and cannot use composition.
- D. Strategy pattern can replace repeated type-based condition chains.

Q226. In production-quality Python code, which statements about Composition Example (Good Case) are valid? Select all that apply.

- A. Cooperative multiple inheritance relies on consistent ``super()`` usage across classes.
- B. Overriding a method should always change input/output contract details.
- C. Depending on abstractions improves testability and implementation swapping.
- D. Multiple inheritance is safe without MRO awareness if class names are unique.

Q227. While applying Building a Real Class Step by Step, which statements are technically correct? Select all that apply.

- A. Polymorphism allows extending behavior without rewriting stable caller logic.
- B. Dependency injection is mainly about reducing lines of code, not testability.
- C. Inheritance automatically guarantees low coupling and high cohesion.
- D. OOP design should preserve clear contracts and minimize unnecessary coupling.

Q228. While applying `__repr__` vs `__str__` (Basic Interview Mention), which statements are technically correct? Select all that apply.

- A. LSP is satisfied as long as child method names match parent method names.
- B. Polymorphism allows extending behavior without rewriting stable caller logic.
- C. Encapsulation protects invariants by exposing behavior methods instead of raw state mutation.
- D. Dependency injection is mainly about reducing lines of code, not testability.

Q229. Which statements are correct about Refactor interview question in Python? Select all that apply.

- A. Dependency injection is mainly about reducing lines of code, not testability.
- B. Strategy pattern can replace repeated type-based condition chains.
- C. Factory-style creation helps isolate construction logic from business workflows.
- D. LSP is satisfied as long as child method names match parent method names.

Q230. Which statements are correct about D - Dependency Inversion Principle (DIP) in Python? Select all that apply.

- A. SRP encourages one primary reason to change per class/component.
- B. Substitutability requires preserving semantic expectations, not just method names.
- C. Protocols require explicit inheritance from the protocol class at runtime.
- D. Inheritance automatically guarantees low coupling and high cohesion.

Q231. Predict the exact single-line output of the following code.

```
class Wallet:
    def __init__(self, balance):
        self._balance = balance
    @property
    def balance(self):
        return self._balance
print(Wallet(13).balance)
```

Answer: _____

Q232. Predict the exact single-line output of the following code.

```
class Base:
    def compute(self): return 5
class Child(Base):
    def compute(self): return super().compute() + 9
print(Child().compute())
```

Answer: _____

Q233. Predict the exact single-line output of the following code.

```
class Base:
    def compute(self): return 2
class Child(Base):
    def compute(self): return super().compute() + 4
print(Child().compute())
```

Answer: _____

Q234. Predict the exact single-line output of the following code.

```
class Email:
    def send(self):
        return 'EMAIL'
class Sms:
    def send(self):
        return 'SMS'
channels = [Email(), Sms()]
print('-'.join(channel.send() for channel in channels))
```

Answer: _____

Q235. Predict the exact single-line output of the following code.

```
class Engine:
    def power(self): return 6
class Car:
    def __init__(self, engine):
        self.engine = engine
    def score(self):
        return self.engine.power() + 5
print(Car(Engine()).score())
```

Answer: _____

Q236. Predict the exact single-line output of the following code.

```
class Base:
    def compute(self): return 8
class Child(Base):
    def compute(self): return super().compute() + 11
print(Child().compute())
```

Answer: _____

Q237. Predict the exact single-line output of the following code.

```
class Engine:
    def power(self): return 4
class Car:
    def __init__(self, engine):
        self.engine = engine
    def score(self):
        return self.engine.power() + 5
print(Car(Engine()).score())
```

Answer: _____

Q238. Predict the exact single-line output of the following code.

```
class Wallet:
    def __init__(self, balance):
        self._balance = balance
    @property
    def balance(self):
        return self._balance
print(Wallet(11).balance)
```

Answer: _____

Q239. Predict the exact single-line output of the following code.

```
class Engine:
    def power(self): return 7
class Car:
    def __init__(self, engine):
        self.engine = engine
    def score(self):
        return self.engine.power() + 5
print(Car(Engine()).score())
```

Answer: _____

Q240. Predict the exact single-line output of the following code.

```
class S:
    def __init__(self): self.n = 10
    def __repr__(self):
        return f'S(value={self.n})'
print(S())
```

Answer: _____

Q241. Predict the exact single-line output of the following code.

```
class Wallet:
    def __init__(self, balance):
        self._balance = balance
    @property
    def balance(self):
        return self._balance
print(Wallet(8).balance)
```

Answer: _____

Q242. Predict the exact single-line output of the following code.

```
class Base:
    def compute(self): return 4
class Child(Base):
    def compute(self): return super().compute() + 3
print(Child().compute())
```

Answer: _____

Q243. Predict the exact single-line output of the following code.

```
class Engine:
    def power(self): return 9
class Car:
    def __init__(self, engine):
        self.engine = engine
    def score(self):
        return self.engine.power() + 5
print(Car(Engine()).score())
```

Answer: _____

Q244. Predict the exact single-line output of the following code.

```
class Base:
    def compute(self): return 6
class Child(Base):
    def compute(self): return super().compute() + 5
print(Child().compute())
```

Answer: _____

Q245. Predict the exact single-line output of the following code.

```
class S:
    def __init__(self): self.n = 5
    def __repr__(self):
        return f'S(value={self.n})'
print(S())
```

Answer: _____

Q246. Predict the exact single-line output of the following code.

```
class Base:
    def compute(self): return 4
class Child(Base):
    def compute(self): return super().compute() + 3
print(Child().compute())
```

Answer: _____

Q247. Predict the exact single-line output of the following code.

```
class Wallet:
    def __init__(self, balance):
        self._balance = balance
    @property
    def balance(self):
        return self._balance
print(Wallet(14).balance)
```

Answer: _____

Q248. Predict the exact single-line output of the following code.

```
class Base:
    def compute(self): return 11
class Child(Base):
    def compute(self): return super().compute() + 4
print(Child().compute())
```

Answer: _____

Q249. Predict the exact single-line output of the following code.

```
class Wallet:
    def __init__(self, balance):
        self._balance = balance
    @property
    def balance(self):
        return self._balance
print(Wallet(9).balance)
```

Answer: _____

Q250. Predict the exact single-line output of the following code.

```
class Engine:
    def power(self): return 9
class Car:
    def __init__(self, engine):
        self.engine = engine
    def score(self):
        return self.engine.power() + 5
print(Car(Engine()).score())
```

Answer: _____

Q251. Predict the exact single-line output of the following code.

```
class S:
    def __init__(self): self.n = 2
    def __repr__(self):
        return f'S(value={self.n})'
print(S())
```

Answer: _____

Q252. Predict the exact single-line output of the following code.

```
class Email:
    def send(self):
        return 'EMAIL'
class Sms:
    def send(self):
        return 'SMS'
channels = [Email(), Sms()]
print('-'.join(channel.send() for channel in channels))
```

Answer: _____

Q253. Which statements are correct about 1 Instance Method vs Class Method vs Static Method in Python? Select all that apply.

- A. Composition is often safer than deep inheritance for change-heavy systems.
- B. OOP design should preserve clear contracts and minimize unnecessary coupling.
- C. Dependency injection is mainly about reducing lines of code, not testability.
- D. Inheritance automatically guarantees low coupling and high cohesion.

Q254. Which statements are correct about Common Smells and Fixes in Python? Select all that apply.

- A. Substitutability requires preserving semantic expectations, not just method names.
- B. Composition is often safer than deep inheritance for change-heavy systems.
- C. Overriding a method should always change input/output contract details.
- D. Multiple inheritance is safe without MRO awareness if class names are unique.

Q255. While applying One-Page Summary, which statements are technically correct? Select all that apply.

- A. Depending on abstractions improves testability and implementation swapping.
- B. Encapsulation is only about making fields private; behavior design is unrelated.
- C. Composition is often safer than deep inheritance for change-heavy systems.
- D. Dependency injection is mainly about reducing lines of code, not testability.

Q256. While applying Inheritance Smells and Refactoring, which statements are technically correct? Select all that apply.

- A. Depending on abstractions improves testability and implementation swapping.
- B. Inheritance automatically guarantees low coupling and high cohesion.
- C. Substitutability requires preserving semantic expectations, not just method names.
- D. Protocols require explicit inheritance from the protocol class at runtime.

Q257. While applying Clean Code Rules for Polymorphic Design, which statements are technically correct? Select all that apply.

- A. Composition is often safer than deep inheritance for change-heavy systems.
- B. Encapsulation protects invariants by exposing behavior methods instead of raw state mutation.
- C. Protocols require explicit inheritance from the protocol class at runtime.
- D. Polymorphism always requires explicit inheritance from a common base class.

Q258. While applying Practice Assignment, which statements are technically correct? Select all that apply.

- A. Polymorphism allows extending behavior without rewriting stable caller logic.
- B. Protocols require explicit inheritance from the protocol class at runtime.
- C. LSP is satisfied as long as child method names match parent method names.
- D. Substitutability requires preserving semantic expectations, not just method names.

Q259. While applying Constructor (__init__) Deep Dive, which statements are technically correct? Select all that apply.

- A. Inheritance automatically guarantees low coupling and high cohesion.
- B. OOP design should preserve clear contracts and minimize unnecessary coupling.
- C. Substitutability requires preserving semantic expectations, not just method names.
- D. LSP is satisfied as long as child method names match parent method names.

Q260. In production-quality Python code, which statements about OOP idea are valid? Select all that apply.

- A. Composition is often safer than deep inheritance for change-heavy systems.
- B. LSP is satisfied as long as child method names match parent method names.
- C. Encapsulation protects invariants by exposing behavior methods instead of raw state mutation.
- D. Protocols require explicit inheritance from the protocol class at runtime.

Q261. Predict the exact single-line output of the following code.

```
class Email:
    def send(self):
        return 'EMAIL'
class Sms:
    def send(self):
        return 'SMS'
channels = [Email(), Sms()]
print('-'.join(channel.send() for channel in channels))
```

Answer: _____

Q262. Predict the exact single-line output of the following code.

```
class Engine:
    def power(self): return 11
class Car:
    def __init__(self, engine):
        self.engine = engine
    def score(self):
        return self.engine.power() + 5
print(Car(Engine()).score())
```

Answer: _____

Q263. Predict the exact single-line output of the following code.

```
class S:
    def __init__(self): self.n = 5
    def __repr__(self):
        return f'S(value={self.n})'
print(S())
```

Answer: _____

Q264. Predict the exact single-line output of the following code.

```
class Wallet:
    def __init__(self, balance):
        self._balance = balance
    @property
    def balance(self):
        return self._balance
print(Wallet(7).balance)
```

Answer: _____

Q265. Predict the exact single-line output of the following code.

```
class Email:
    def send(self):
        return 'EMAIL'
class Sms:
    def send(self):
        return 'SMS'
channels = [Email(), Sms()]
print('-'.join(channel.send() for channel in channels))
```

Answer: _____

Q266. Predict the exact single-line output of the following code.

```
class S:
    def __init__(self): self.n = 6
    def __repr__(self):
        return f'S(value={self.n})'
print(S())
```

Answer: _____

Q267. Predict the exact single-line output of the following code.

```
class Engine:
    def power(self): return 6
class Car:
    def __init__(self, engine):
        self.engine = engine
    def score(self):
        return self.engine.power() + 5
print(Car(Engine()).score())
```

Answer: _____

Q268. While applying Refactor interview question, which statements are technically correct? Select all that apply.

- A. Composition is often safer than deep inheritance for change-heavy systems.
- B. Factory-style creation helps isolate construction logic from business workflows.
- C. Design patterns are mandatory in all modules, even for trivial scripts.
- D. OOP design should preserve clear contracts and minimize unnecessary coupling.

Q269. In production-quality Python code, which statements about Way B: Duck Typing (no inheritance, same method shape) are valid? Select all that apply.

- A. OOP design should preserve clear contracts and minimize unnecessary coupling.
- B. Depending on abstractions improves testability and implementation swapping.
- C. Protocols require explicit inheritance from the protocol class at runtime.
- D. Composition is often safer than deep inheritance for change-heavy systems.

Q270. In production-quality Python code, which statements about Where to use are valid? Select all that apply.

- A. Factory-style creation helps isolate construction logic from business workflows.
- B. Design patterns are mandatory in all modules, even for trivial scripts.
- C. OOP design should preserve clear contracts and minimize unnecessary coupling.
- D. Strategy pattern can replace repeated type-based condition chains.

Q271. Which statements are correct about Python snippet in Python? Select all that apply.

- A. Strategy pattern requires inheritance and cannot use composition.
- B. Factory-style creation helps isolate construction logic from business workflows.
- C. Strategy pattern can replace repeated type-based condition chains.
- D. Substitutability requires preserving semantic expectations, not just method names.

Q272. Predict the exact single-line output of the following code.

```
class Engine:
    def power(self): return 3
class Car:
    def __init__(self, engine):
        self.engine = engine
    def score(self):
        return self.engine.power() + 5
print(Car(Engine()).score())
```

Answer: _____

Q273. Predict the exact single-line output of the following code.

```
class Base:
    def compute(self): return 9
class Child(Base):
    def compute(self): return super().compute() + 5
print(Child().compute())
```

Answer: _____

Q274. Predict the exact single-line output of the following code.

```
class Base:
    def compute(self): return 4
class Child(Base):
    def compute(self): return super().compute() + 10
print(Child().compute())
```

Answer: _____

Q275. Predict the exact single-line output of the following code.

```
class Engine:
    def power(self): return 11
class Car:
    def __init__(self, engine):
        self.engine = engine
    def score(self):
        return self.engine.power() + 5
print(Car(Engine()).score())
```

Answer: _____

Q276. In production-quality Python code, which statements about 4 Constant naming are valid? Select all that apply.

- A. Clean code decisions should optimize maintainability and testability, not cleverness.
- B. Long methods are preferred because function calls always reduce readability.
- C. Guard clauses can improve readability by reducing deep nesting.
- D. Magic numbers improve maintainability by reducing named constants.

Q277. While applying Business impact of poor code, which statements are technically correct? Select all that apply.

- A. Catching broad exceptions and ignoring them is clean because execution continues.
- B. Refactoring should be done in small safe steps with behavior checks.
- C. Clean code is stronger when intent-revealing names and focused responsibilities are used.
- D. Magic numbers improve maintainability by reducing named constants.

Q278. In production-quality Python code, which statements about Logging and Observability as Clean-Code Concerns are valid? Select all that apply.

- A. Long methods are preferred because function calls always reduce readability.
- B. Refactoring should be done in small safe steps with behavior checks.
- C. Clean code is stronger when intent-revealing names and focused responsibilities are used.
- D. Catching broad exceptions and ignoring them is clean because execution continues.

Q279. While applying Bad, which statements are technically correct? Select all that apply.

- A. Guard clauses can improve readability by reducing deep nesting.
- B. Refactoring should be done in small safe steps with behavior checks.
- C. Copy-pasting similar logic is better than creating shared abstractions.
- D. Magic numbers improve maintainability by reducing named constants.

Q280. Predict the exact single-line output of the following code.

```
def normalize_name(raw_name):
    return raw_name.strip().lower().replace(' ', '_')
print(normalize_name(' Clean Code '))
```

Answer: _____

Q281. Predict the exact single-line output of the following code.

```
def pick_primary(configured, fallback):
    if configured is None:
        return fallback
    return configured
print(pick_primary(None, 2))
```

Answer: _____

Q282. Predict the exact single-line output of the following code.

```
def compute_total(unit_price, quantity):  
    return unit_price * quantity  
print(compute_total(4, 5))
```

Answer: _____

Q283. Which statements are correct about Refactoring Safety Net (Production) in Python? Select all that apply.

- A. Copy-pasting similar logic is better than creating shared abstractions.
- B. Magic numbers improve maintainability by reducing named constants.
- C. Guard clauses can improve readability by reducing deep nesting.
- D. Clean code is stronger when intent-revealing names and focused responsibilities are used.

Q284. Predict the exact single-line output of the following code.

```
def normalize_name(raw_name):  
    return raw_name.strip().lower().replace(' ', '_')  
print(normalize_name(' Clean Code '))
```

Answer: _____

Q285. Predict the exact single-line output of the following code.

```
def normalize_name(raw_name):  
    return raw_name.strip().lower().replace(' ', '_')  
print(normalize_name(' Clean Code '))
```

Answer: _____

Q286. In production-quality Python code, which statements about ■ Step 5: Function behavior are valid? Select all that apply.

- A. Assignment copies objects by value in Python for all data types.
- B. `try/finally` skips `finally` when the `try` block returns early.
- C. Fundamentals should be reasoned using Python's binding and execution model, not memorized guesses.
- D. LEGB describes Python name lookup order across scopes.

Q287. Predict the exact single-line output of the following code.

```
base = 3  
alias = base  
base = base + 6  
print(base - alias)
```

Answer: _____

Q288. In production-quality Python code, which statements about Context Manager Control Flow (with) and Cleanup Guarantees are valid? Select all that apply.

- A. Function defaults are evaluated at definition time, so mutable defaults can retain state.
- B. `try/finally` skips `finally` when the `try` block returns early.
- C. Loop `else` executes only when the loop completes without `break`.
- D. Mutable default arguments are recreated fresh on each function call.

Q289. In production-quality Python code, which statements about if/elif/else and Boolean Precedence are valid? Select all that apply.

- A. Context managers (`with`) are preferred for deterministic resource cleanup.
- B. `is` checks identity while `==` checks value equality semantics.
- C. The `match/case` default branch is required and must be listed first.
- D. The `finally` block runs only when no exception occurs in `try`.

Q290. Which statements are correct about Iterator and Generator Flow Thinking in Python? Select all that apply.

- A. `is` checks identity while `==` checks value equality semantics.
- B. The `finally` block runs only when no exception occurs in `try`.
- C. Mutable default arguments are recreated fresh on each function call.
- D. Function defaults are evaluated at definition time, so mutable defaults can retain state.

Q291. In production-quality Python code, which statements about ■ Step 5: Function call are valid? Select all that apply.

- A. LEGB describes Python name lookup order across scopes.
- B. Function defaults are evaluated at definition time, so mutable defaults can retain state.
- C. Assignment copies objects by value in Python for all data types.
- D. `try/finally` skips `finally` when the `try` block returns early.

Q292. Predict the exact single-line output of the following code.

```
left = 9
right = 12
left, right = right, left
print(left + right)
```

Answer: _____

Q293. Predict the exact single-line output of the following code.

```
left = 8
right = 7
left, right = right, left
print(left + right)
```

Answer: _____

Q294. Predict the exact single-line output of the following code.

```
try:
    value = 11
    print(value)
finally:
    pass
```

Answer: _____

Q295. Predict the exact single-line output of the following code.

```
numbers = [3, 6, 1]
average = sum(numbers) // len(numbers)
print(average)
```

Answer: _____

Q296. Predict the exact single-line output of the following code.

```
numbers = [6, 10, 7]
average = sum(numbers) // len(numbers)
print(average)
```

Answer: _____

Q297. Which statements are correct about Mental Model in Python? Select all that apply.

- A. Function defaults are evaluated at definition time, so mutable defaults can retain state.
- B. Assignment copies objects by value in Python for all data types.
- C. `is` checks identity while `==` checks value equality semantics.
- D. `try/finally` skips `finally` when the `try` block returns early.

Q298. Predict the exact single-line output of the following code.

```
try:
    value = 8
    print(value)
finally:
    pass
```

Answer: _____

Q299. Predict the exact single-line output of the following code.

```
def scale(value, factor):
    return value * factor
print(scale(11, 4))
```

Answer: _____

Q300. Which statements are correct about ■ Step 4: Operation type in Python? Select all that apply.

- A. Context managers (`with`) are preferred for deterministic resource cleanup.
- B. Fundamentals should be reasoned using Python's binding and execution model, not memorized guesses.
- C. Function defaults are evaluated at definition time, so mutable defaults can retain state.
- D. Importing a module never executes its top-level statements.